

## Какие бывают: Составные типы

### Работа с составными типами

#### 1. Определение

1. тип ( `struct` , `class` , ...)
2. имя
3. компоненты
4. ключи доступа ( `public` , `private` , ..)

#### 2. Создать переменную этого типа

#### 3. Использовать ( `.` , `::` , `->` )

### Enum

**Enum** - набор именованных целочисленных констант

```
enum class Имя_перечисления (: базовый тип) {  
    константа_1 = значение_1,  
    константа_2 = значение_2,  
    константа_3 = значение_3  
}
```

Если значение константы не указано, то ее значение будет равно предыдущему значению + 1

```
enum class Mine : short {  
    a, // 0  
    b, // 1 (a + 1)  
    c // 2 (b + 1)  
}
```

```
Mine::a // 1
```

# Struct

**struct** - позволяет объединить под один именем несколько элементов данных разных типов

## Структуры в стеке

```
// Объявление типа, память не выделяется
struct Abonent {
    string name { "noname" };
    double fee { 100.0 };
    double tariff;
    int minutes;

    void read_abonent(); // чтение данных
    double get_cost(); // расчет суммы
};

void Abonent::read_abonent() {
    ...
    cout << "...";
    cin >> fee; // т.к. указано, что эта функция из
    структуры, то в fee все корректно подставится
}

double Abonent::get_cost() {
    ...
}

int main() {
    // Здесь уже выделяется память в стеке
    // Все параметры передаются в порядке объявления
    Abonent a{"Bob", 200.1, 1.5, 123};

    // Тут можно в любом порядке
    Abonent a{
```

```

        .name = "Bob",
        ...
    }

    cout << a.name; // Bob
    cout << a.get_cost();
    return 0;
}

```

## Структуры в куче

```

int main() {
    Abonent* a_p = new Abonent {...};

    // Можно так
    cout << (*x).name // Bob

    // Но лучше так
    cout << x->name // Bob

    delete x;
    return 0;
}

```

## Массивы структур

```

int main () {
    Abonent arr[3];
    for (Abonent& a: arr) {
        a.read_abonent();
    }
    return 0;
}

```

```

int main () {
    Abonent* arr = new Abonent[3];
    for (size_t i = 0; i < 3; i++) {
        arr[i].read_abonent();
    }
    return 0;
}

```

## Структуры в структурах

```

// Тут структура снаружи
struct Date {
    int day;
    int month;
    int year;
};

struct Abonent {
    ...
    Date date {1, 2, 3};
}

Abonent a = {...}
cout << a.date.day // 1

```

```

struct Abonent {
    struct Date {
        int day;
        int month;
        int year;
    } date {1, 2, 3};
}

Abonent a = {...}

```

```
cout << a.date.day // Ошибка
```

Экономим память: битовые поля

```
// Это плохо с точки зрения памяти – для даты столько  
не надо  
struct Date {  
    int day;  
    int month;  
    int year;  
};  
  
struct Date {  
    unsigned short day : 5; // 5 байт –  $2^5=32$   
значения  
    unsigned short month : 4; // 4 байт –  $2^5=16$   
значений  
    unsigned short year;  
};
```

Дополнительно про структуры

1. Можно управлять доступом
2. Можно делать конструкторы
3. Структуры можно наследовать

Union

**union** - составной тип, все элементы которого расположены по одному и тому же адресу (похож на struct)

Используется, если очень мало памяти, есть особенности с доступом

```
union Types {  
    short i;  
    long l;  
    float f;  
}
```